MARK: Product and Build Guide

Oct 27, 2026 · @Founder's Office

What MARK is

MARK reads every document in an AICTE approval application, proves each one is real, checks the file against the Approval Process Handbook, finds faculty who are claimed by more than one institution, and shows the reviewer exactly why. It sits between the institution's upload and the reviewer's desk, so a reviewer opens a file that has already been read, scored and explained.

The problem it answers: every approval cycle AICTE receives thousands of applications (Extension of Approval, New Institution, Increase in Intake, Closure of Course), each a bundle of affidavits, land deeds, building plans, faculty lists, CA certificates, bank statements, lab and library registers, geotagged photos and university NOCs. Scrutiny is manual, slow and uneven. A single file cannot show that the same professor is on three payrolls, that an affidavit was edited after notarisation, or that a deed describes land that is half the size on the ground.

The name: MARK is the mark an application earns when it is real. As a backronym, Multi-source Authentication and Regulatory Knowledge.

Alignment with the problem statement

The problem statement asks for AI-powered application validation, document authentication and regulatory compliance verification. MARK maps each part to an engine, and adds a fourth that the parts need but do not name.

Problem statement part	MARK engine	What it does	Where you see it
Application validation	Validation	Classifies every upload, extracts fields, checks completeness, dates in cycle, and that every number agrees across documents	Review desk, application file (Overview, Documents), pre-check

Problem statement part	MARK engine	What it does	Where you see it
Document authentication	Authentication	Image forensics (error-level, copy-move, font breaks, metadata, synthetic-image score), registry checks (UDIN, EPFO, DigiLocker, land records, NOC register), satellite footprint and photo GPS	Document viewer with forensic lens, application flags
Regulatory compliance verification	Compliance	The Approval Process Handbook as versioned rules with clause numbers; each norm passes or fails with the actual value and the required value	Handbook tab on a file, Handbook rules page
Needed by all three, not named	Faculty graph	Matches PAN, date of birth, degree number and face embedding across every application in the cycle; finds people on two or more faculty lists and the EPFO trail that says who really pays them	Faculty graph page, Faculty tab on a file

Everything the engines find becomes one trust score per application with an evidence link on every point, a deficiency letter that cites clauses, a copilot that answers only from the file, and a tamper-evident audit chain of every machine and human decision.

Language rule across the product: MARK never says fraud. It says discrepancy, unverified, or needs a fresh copy. A flag is a question for a human, not a verdict.

How MARK works

A file goes through ten steps in about 20 to 30 seconds and comes out as one trust score, four engine scores, a list of flags with evidence, and a recommendation.

The four engines

Engine	Question it answers	Inputs	Typical flags
Validation	Is the file complete and consistent with itself?	Classified documents, extracted fields, dates	Missing mandatory document, register older than 60 days, lab count differs between inventory and photos
Authentication	Is each document real?	Forensic scores, registry responses, satellite and EXIF	Copy-move on a notary seal, UDIN not in the ICAI register, deed area 2.3x the satellite footprint, synthetic lab photos
Compliance	Does the file meet the handbook?	Extracted claims against versioned rules	Faculty ratio 1:31 against 1:20, built-up area below 10 sqm per student
Faculty graph	Is every faculty member real, present and only here?	PAN, DOB, degree, face embedding, EPFO trail across all applications	6 people claimed full-time at 3 institutions in the same semester

The trust score

Every flag carries a severity (critical, major, minor, info), the engine that raised it, a point cost, the evidence sentence, and where possible the handbook clause and the document it points to. The score starts at 100 and subtracts each flag's points, weighted by engine: faculty 1.3, authentication 1.25, compliance 1.1, validation 1.0. Each engine also keeps its own score (100 minus 2.2 points per flag point) so the reviewer sees which engine is unhappy. The score is clamped between 4 and 99.

Bands drive the desk: 85 and above is fast-track (sign and move on), 60 to 84 is second look, under 60 is hold for senior review. The thresholds are settings, not code.

The reviewer can confirm or dismiss any flag. A dismissed flag drops out of the score immediately, and the ring, the engine bars and the deficiency letter all follow. The dismissal is logged to the audit chain.

The pipeline, step by step

1. Intake: documents arrive from the portal, DigiLocker or a drag-and-drop (about 0.4 s).
2. Classify: each upload is named by what it is, not by its filename (1.1 s).
3. Read: OCR in English, Hindi and regional scripts, tables kept intact (3.8 s).
4. Extract: names, areas, amounts, dates, UDINs, survey numbers, with page and box for each (1.9 s).
5. Cross-check: every number compared with every other document (0.9 s).
6. Forensics: error-level analysis, copy-move, font profiling, metadata history, synthetic-image detection (2.6 s).
7. Registries: DigiLocker, ICAI UDIN, EPFO, National Academic Depository, state land records, Bhuvan satellite, university NOC register (5.2 s, the slowest step because it waits on external systems).
8. Faculty graph: PAN, DOB, face and timeline matched across every institution in the cycle (2.1 s).
9. Handbook: fourteen norms as code, each with its clause (0.6 s).
10. Trust score: weighted, explained, evidence-linked (0.2 s).

The Pipeline tab on any file replays these steps at four times speed so a reviewer can see what was checked and how long each step took.

Using MARK, page by page

MARK runs at http://localhost:3900. Every inner page has a back button top left, a tab strip with every section, and a Run a pre-check button top right. Press Enter on the opening page to go straight to the review desk.

Opening page (/)

A drop of ink blooms into a web of points behind the name, breathes, folds back and blooms again. The name settles, the seal takes the corner of the K, the tagline lands and the door opens. Three ways in: Open the review desk, Pre-check an institution's file, See the faculty graph. Nothing on this page follows the cursor.

Review desk (/review)

The reviewer's morning. The ledger lists every application sorted by risk, worst first. Each row shows the trust ring, the institution, the application type and id, flag dots (red critical, amber major, grey minor), the assigned reviewer and the status.

- Band tabs: All files, Hold (under 60), Second look (60 to 84), Fast-track (85+)

- Search by name, city or application id; filter by state; sort by risk, newest or name
- Right rail: the file to start with, the four engines explained, the ghost-faculty count with a link to the graph
- Click any row to open the file

Application file (/review/APP-…)

The heart of the product. The header gives status, type, id, submission date, reviewer, the institution and its details, and MARK's one-paragraph summary. The trust ring on the right shows the score and the verdict stamp (hold, second look, fast-track). Three actions sit under the summary: Recommend, Send deficiency letter, Refer to Expert Visit Committee. Each stamps a verdict and ripples the background.

Four engine cards show each engine's score, bar and flag count. Below them, seven tabs:

1. Overview: flags worst first, each with severity, engine, point cost, title, evidence, clause and a link to the page it points at. Confirm or Dismiss on every flag; the score re-computes. Beside them, what the file claims (land, built-up area, intake, faculty claimed and verified, labs, library, funds, fees) and the document list with status.
2. Documents: one card per document with status, pages, upload date, every registry check, and the three forensic gauges (error-level, copy-move, synthetic).
3. Faculty: everyone on the list with designation, department, degree, masked PAN, EPFO trail, and a red badge naming the other institutions that also claim them.
4. Handbook: every active norm evaluated against the file, with clause, the file's actual value, the required value and pass or fail.
5. Pipeline: the ten-step replay with timings.
6. Copilot: ask the application. Starter questions cover faculty, clauses, the letter and what would make the file fast-track. Answers cite the document, registry or clause.
7. Deficiency letter: generated from the major and critical flags that are not dismissed, each naming the document, the clause and what to send. Print or save as PDF, or send through the portal.

Document viewer (/review/APP-…/doc/…)

The document rendered as paper, so the reviewer sees the page MARK read. Red boxes mark what the engines found (a cloned notary seal, a UDIN not in the register, a survey number that changed hands, photos that look synthetic). Move the pointer over the page to look through the forensic lens; switch layers with the pills: Original, Error level, Copy-move, Metadata. The right rail gives the verdict stamp, every check with its source and

detail, the three gauges against their thresholds, the flags on this document, and three reviewer actions: request a fresh copy from the issuer, mark as verified by eye, escalate to a senior reviewer.

Faculty graph (/graph)

A force-directed graph of the whole cycle. Squares are institutions, dots are faculty, red dots and red lines are people on two or more lists. Hover to read a label, click to pin it in the side panel. Toggle Whole country or Only the rings, Freeze or Resume the layout. The side panel lists every ring found this cycle with the people and the application ids involved.

Handbook rules (/rules)

The Approval Process Handbook as code. Each rule shows its id, clause, category, status (active, draft, retired), version, plain-words norm, the expression it evaluates, and how many files it fails on. Filter by category. On the right, pick a clause, press Draft the rule, watch the rule appear, then approve it for the cycle. A human approves every rule before it runs.

Pre-check (/precheck)

The institution's side. Drop documents or click to choose, or load the sample set of nine. MARK classifies each file by name, lists what is missing against the mandatory set, then runs the same pipeline the reviewer uses. The result is a readiness score, a verdict stamp and a list of findings with what to fix before submitting. Nothing is stored or sent.

National map (/map)

A tile-grid map of India. Each state is a tile; its ink is the average trust of its files, a red dot marks ghost faculty. Click a tile for its files, average and ghost count, then open any file. Trust by institution type and the weekly trend sit alongside.

Virtual inspection (/inspect)

A guided phone walkthrough of seven stops: gate, classrooms, computer lab, workshop, library, faculty room, hostel. Press play and MARK counts rooms, seats, terminals and shelves against the file. Stops that differ from the file turn red and land in the findings panel for the Expert Visit Committee. GPS lock, liveness and frame hashing stop staging.

Audit chain (/audit)

Every pipeline result and reviewer click as an entry hashed to the one before it with SHA-256, computed in the browser. Verify the chain, then press Tamper with entry 5 to change a score and watch every later link fail. Reset restores it.

Settings (/settings)

Role (Reviewer, Senior reviewer, Regional officer, Institution, Administrator), thresholds (fast-track, hold, face match, error-level), integrations with on and off switches, and privacy controls (mask Aadhaar and PAN, on-premise only, bias guard, retention).

Why MARK (/about)

The pitch in six cards, the architecture in eight steps, and the five-minute walkthrough below.

Five-minute walkthrough

The story is one file: Shree Vallabh Institute of Technology, Indore, application APP-2026-0417, Extension of Approval. Six people on its faculty list are full-time at two other colleges, the affidavit backing the list was edited after the notary, and the CA certificate carries a UDIN the ICAI register does not know. Trust score 41.

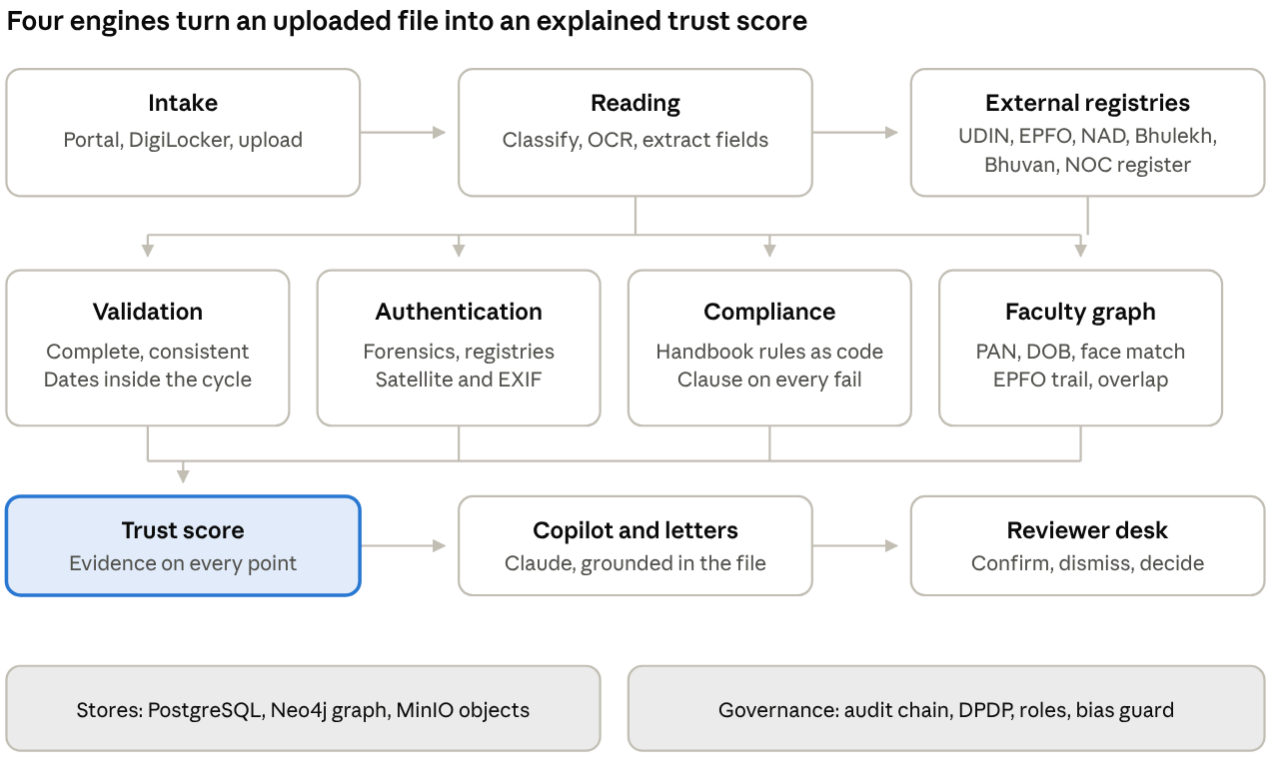
1. Open the app. Let the name settle, then press Enter. Say: every approval cycle, AICTE reads lakhs of pages by hand; some of the faculty on those pages work at three colleges at once.
2. Review desk. Point at the ledger: forty-eight files arrived this week, sorted by risk, the holds at the top. Clean files wait at the bottom with a recommendation ready to sign. Click the Indore row.
3. The file. Read the summary aloud. Point at the ring (41, hold) and the four engines: Faculty and Authentication are red, Validation and Compliance are fine. Say: the score is explained, not asserted.
4. Overview tab. The first flag: six faculty claimed full-time at three institutions in the same semester. Press Confirm; the background ripples red. Scroll to the minor flag about lab count and press Dismiss; the score climbs. Say: the reviewer stays in charge, the machine proposes.
5. Open the page. On the affidavit flag press Open the page. Hover the notary block to look through the lens, then press Error level. The seal glows: a pixel-level clone of page one, Photoshop in the metadata six days after notarisation. Press Request a fresh copy from the issuer.

6. Faculty graph. Back, then the Faculty graph tab. Press Only the rings. Click a red dot: Dr. Anil Kumar Mishra, on three lists, no EPFO trail at any of them. Say: a single file cannot see this; the graph can.
7. Handbook tab on the file. Clause 4.5 fails at 1:31 against 1:20 once the unverified faculty are removed. Say: every fail cites a paragraph of the handbook.
8. Deficiency letter tab. The letter is already written: three items, each with the document, the clause and what to send. Press Print or save as PDF.
9. Refer to Expert Visit Committee. The stamp lands. Open the Audit chain tab, press Verify the chain, then Tamper with entry 5, then Verify again. The chain breaks at entry five. Say: nothing in MARK is unsigned.
10. Close on Pre-check. Load the sample set, run it. Say: the institution would have caught most of this before submitting, which is fewer letters for them and a calmer morning for the reviewer.

Two contrast files if there is time: Coastal College of Pharmacy, Visakhapatnam (APP-2026-0388) where the deed claims 2.5 acres and the satellite sees 1.1, and Nalanda School of Management, Patna (APP-2026-0402), a clean file recommended in nineteen seconds.

Architecture and technology stack

MARK is one Next.js application today: the pages, the scoring, the rules, the graph and the copilot route all live in one codebase, with the data layer written so each engine can move to its own service without changing the pages.



MARK architecture · intake, four engines, score, governance

A file is read once, every engine sees the same extracted fields and registry responses, and the score, the copilot and the reviewer's desk all read from one evidence-linked result. The stores and the governance layer sit under everything.

What is in the build now

Layer	Technology	Used for
Framework	Next.js 16 (App Router, Turbopack), React 19, TypeScript 5	Every page and the copilot API route
Styling	Tailwind CSS 4 with design tokens in globals.css	Paper and ink theme, sheets, buttons, badges, stamps
Motion	Motion 14 (Framer Motion)	Page transitions, reveals on mount, score ring, bars, stamps, tab switches
Timelines	GSAP 3	Counters, the pipeline strip, the inspection walkthrough
3D	Three.js 0.186 with custom GLSL point and line shaders	The ink web on the opening page

Layer	Technology	Used for
Fonts	Bricolage Grotesque (display), Instrument Sans (body), IBM Plex Mono (labels), loaded through next/font	Typography
Canvas	2D canvas	Background ledger lines and ripples, the force-directed faculty graph, the tile map uses CSS grid
Crypto	Web Crypto SHA-256	The audit chain, computed and verified in the browser
Language model	Anthropic SDK, Claude Sonnet 5.5 by default	Reviewer copilot and letters, grounded in the file; falls back to a scripted engine over the same data without a key
Data	Deterministic seeded generator in lib/data.ts	48 institutions, 48 applications, about 700 faculty, documents, flags, claims
Build and run	npm, next build, next start on port 3900, run.sh	Local production build

Intended production integrations

System	Role in MARK	Engine
AICTE approval portal	Pulls applications, pushes recommendations and letters	Intake, output
DigiLocker	Issued documents straight from the issuer	Authentication
ICAI UDIN register	Verifies every CA certificate	Authentication
EPFO	Contribution trail: who actually pays each faculty member	Faculty graph
National Academic Depository	Degree certificates by number	Faculty graph
State land records (Bhulekh)	Survey numbers and ownership	Authentication
Bhuvan satellite imagery	Campus footprint against the deed	Authentication

System	Role in MARK	Engine
University NOC registers	Affiliation and NOC validity	Authentication
MCA and Registrars of Societies	Trusts and societies behind institutions	Validation

Models planned for production

Task	Candidate
OCR and layout, Indic scripts	Surya or PaddleOCR with a layout model that keeps tables
Document classification and field extraction	A LayoutLM-style model fine-tuned on AICTE document types
Image forensics	Error-level analysis, SIFT-based copy-move, font profiling, a synthetic-image detector
Faces	InsightFace embeddings, threshold 0.9
Graph	PostgreSQL plus Neo4j for the national faculty graph
Queue and storage	Celery workers, MinIO object store
Language	Claude for clause-to-rule drafts, the copilot and letters, always grounded in the file
Identity and access	Keycloak with roles matching the settings page

Data model and scoring

All data is synthetic and deterministic: a seeded random generator (seed 26261) builds the same world on the server and in the browser, so pages render identically on both and the demo never changes between runs.

Entities

Entity	Key fields	Where it lives
Institution	id, name, state, city, type (Engineering, Pharmacy, Management, Polytechnic, Architecture), established, intake, application id, trust, NAAC grade, NBA	lib/data.ts
Application	id, institution, kind, cycle, submitted date, status, score, breakdown per engine, flags, documents, claims, reviewer, summary, pipeline time	lib/data.ts
Document	id, type, file, pages, upload date, status, checks (one per source), forensic scores (error-level, copy-move, font breaks, metadata, synthetic)	lib/data.ts
Flag	id, severity, engine, title, evidence, clause, document id, points	lib/data.ts
Faculty	id, name, PAN, DOB, degree, designation, department, institution ids, EPFO trail, photo seed, joined	lib/data.ts
Claims	land on deed and on satellite, built-up area, intake, faculty claimed and verified, labs and labs required, library volumes, bank balance, classrooms, fees collected and approved	lib/data.ts
Rule	id, clause, title, norm, category, version, status, expression, evaluate function	lib/rules.ts
Audit entry	sequence, time, actor, action, application id, detail, previous hash, hash	lib/audit.ts

How the world is generated

Three files are written by hand because they carry the story: Indore (ghost ring, edited affidavit, missing UDIN), Visakhapatnam (land mismatch, synthetic photos, built-up area below norm) and Patna (clean). Two more share a faculty ring in Madhya Pradesh and two in Tamil Nadu share a smaller pair. The remaining institutions are generated with Indian names, cities by state, realistic intakes, and a few random minor or major flags so the ledger has texture. Faculty are generated per institution; ring members are appended to several institutions' lists so the graph finds them.

From flags to a score

```
\text{score} = \operatorname{clamp}\left(100 - \sum_{f} p_f \cdot w_{e(f)},\
4,\ 99\right)
```

where p is the flag's points and w is the engine weight (faculty 1.3, authentication 1.25, compliance 1.1, validation 1.0). Each engine's own score is 100 minus 2.2 times the points of its flags, floored at 0. Dismissed flags are removed from the sum before it is computed.

Rules as code

Each rule is a small object with a clause number, a plain-words norm, a readable expression and an evaluate function that takes the application's claims and returns pass or fail with the actual and required values. Rules carry a version and a status: active rules run, draft rules are shown for the next cycle, retired rules are kept so last year's file is judged by last year's book. The rules page demonstrates a language model drafting a rule from a clause and a human approving it.

The audit chain

Each entry's hash is SHA-256 over the previous hash, the time, the actor, the action, the application id and the detail. The first entry chains to sixty-four zeros. Verification recomputes every hash in order and compares; changing one character in any entry breaks every later link. In production the nightly head hash can be anchored to a public ledger.

Building and running MARK

MARK needs Node.js 20 or newer and npm. The code lives in the app folder of the MARK project directory.

Commands

```
cd app
npm install
npm run dev          # development server with hot reload,
http://localhost:3900
npm run build        # production build
npm run start        # serve the production build on port 3900
./run.sh             # install if needed, then build and start
npx eslint .         # lint
npx tsc --noEmit     # type check
```

Use the production build for a demo: the opening scene and the counters run smoother than in development mode.

Environment variables

Copy .env.example to .env.local in the app folder. Both variables are optional.

Variable	Effect
ANTHROPIC_API_KEY	With a key, the copilot and letters are written by Claude, grounded in the file. Without it, the copilot answers from a scripted engine that reads the same data.
MARK_MODEL	Model id for the copilot. Default claude-sonnet-5-5.

Folder structure

Path	Holds
app/page.tsx	The opening page
app/layout.tsx	Fonts, background, shell
app/globals.css	Design tokens, sheets, buttons, badges, stamp, grain
app/review/page.tsx	Review desk
app/review/[id]/page.tsx	Application file with its seven tabs
app/review/[id]/doc/[docId]/page.tsx	Document viewer
app/graph, app/rules, app/precheck, app/map, app/inspect, app/audit, app/settings, app/about	One page each
app/api/copilot/route.ts	Copilot API: Claude with fallback
components/Shell.tsx	Header, back button, tab strip, page transitions, footer
components/Background.tsx	Paper environment: ledger lines, light pools, ripples (pulse event)
components/InkWeb.ts	The Three.js ink web on the opening page

Path	Holds
components/Logo.tsx	The wordmark as SVG strokes and the seal
components/ui.tsx	Panel, Kicker, Reveal, PageHead, Stat, ScoreRing, Counter, Bar, Stamp, Severity, Sparkline
components/AppRow.tsx, Pipeline.tsx, Copilot.tsx, DocPaper.tsx	Ledger row, pipeline strip, chat, document renderer
lib/types.ts	Every type
lib/data.ts	The seeded world, scoring, helpers
lib/rules.ts	Handbook rules as code, clause samples
lib/audit.ts	Audit entries, SHA-256 chain, verification
public/brand/favicon.svg	Favicon

Where to change things

- A new hero file: add a makeInstitution call in lib/data.ts with its flags, claims and document checks. Hand-written institutions come before the random loop.
- A new handbook rule: add an object to the rules array in lib/rules.ts with clause, norm, expression and evaluate.
- Score weights and bands: ENGINE_WEIGHT in lib/data.ts; the bands are in the review desk and the settings page.
- Navigation: the LINKS array in components/Shell.tsx.
- Theme: the tokens at the top of app/globals.css; every component reads colours from them.
- Pipeline steps and timings: STEPS in components/Pipeline.tsx.
- The ripple on a stamp: call pulse(colour, x, y) from components/Background.tsx anywhere.

Deploying

The app builds as a standard Next.js project and runs anywhere Node runs. For AICTE it is meant to run on-premise behind the Council's identity provider; a container image with next start is enough. Vercel also works for a hosted demo.

Design system

The world is a government file room: warm manila paper, an ink-blue pen, a stamp seal. The canvas is never pure white or pure black; it is paper, with grain and faint ledger lines that bend when something is stamped.

Colours

Token	Value	Used for
paper	#efe8da	Canvas
ink	#1c2541	Text, primary buttons, the wordmark
accent (official ink)	#2f45c4	Call to action, focus rings, the seal, rare emphasis
seal	#b23a2f	Discrepancies, critical flags, hold stamps
ok	#2e7d5b	Verified, fast-track, pass
warn	#b8760f	Second look, re-check
sheet	rgba(255,252,245,0.74)	Translucent panels with blur over the paper

The accent appears in rare doses so it still reads as emphasis. Status colours carry meaning everywhere: green verified, amber re-check, red discrepancy.

Type

Bricolage Grotesque for display (large headings and the wordmark, optical size axis on), Instrument Sans for body, IBM Plex Mono for kickers, ids, hashes and small labels. Kickers are uppercase mono at 0.68 rem with wide tracking.

Motion rules

- Nothing animates on scroll. Everything plays on mount or on an action, so a demo never depends on scrolling.
- Page transitions fade with a slight blur; reveals rise 14 px with blur over 0.7 s on one easing curve.
- Numbers count up (GSAP); rings and bars draw to their value (Motion).
- A verdict stamps in from 1.6x scale with a rubber-stamp mask and ripples the background through the pulse event.
- The opening scene is a 17 s loop; it pauses when the tab is hidden and shows a still drop under reduced motion or without WebGL.

- Nothing on the opening page follows the cursor.

Copy rules

- Never the word fraud. Use discrepancy, unverified, needs a fresh copy, needs verification.
- Every flag is a question for a human, not a verdict; the product says so where it matters.
- Plain words, short sentences, no corporate filler. Headlines make a claim ("One person, three payrolls").
- No hackathon, team, prototype or placeholder wording anywhere in the product.

Accessibility

Skip link to main content, semantic landmarks, visible focus rings in the accent, 44 px touch targets on coarse pointers, decorative canvases and SVGs marked aria-hidden, reduced-motion respected throughout, body text contrast above 4.5:1 on paper.

Governance, privacy and fairness

MARK proposes; a person decides; the record shows both. That is the rule behind every screen.

- Human in the loop: no recommendation, deficiency letter or referral leaves without a reviewer's action. Confirm and dismiss on every flag; the score follows the reviewer, and the audit chain records who changed what.
- Explainability: every point of the score links to a document, a registry response or a handbook clause. A score that cannot say which clause it used is an opinion; MARK's can.
- Audit: every machine result and every human click is a hashed entry; the chain is verifiable by anyone with the log and can be anchored to a public ledger nightly.
- Privacy (Digital Personal Data Protection Act, 2023): purpose-bound processing, Aadhaar and PAN masked at rest and on screen, raw uploads retained for a set period then deleted, consented and logged registry calls, on-premise models so no document leaves the Council's network.
- Fairness: the scoring engine never sees the institution's name, state or owner; only documents, registry responses and norms. Thresholds are tuned from reviewer feedback and reviewed per cycle.
- Roles: Reviewer, Senior reviewer (can override a score with a signed reason), Regional officer (map and queue for their states), Institution (pre-check and own files only), Administrator (rules, integrations, thresholds; cannot touch a score).

- Language: discrepancy, not fraud. A flag opens a question; the institution gets a letter that names the document, the clause and what to send.

Known limits and next steps

What is real in the current build: the pages, the scoring and dismissal logic, the handbook rules engine, the force graph, the SHA-256 audit chain, the document renderer with forensic layers, the pipeline replay, the pre-check classifier by filename, and the copilot route with a Claude path.

What is simulated: every institution, person, document and registry response is generated in lib/data.ts. Registry calls, satellite lookups, OCR, forensics and face matching return pre-computed results. The virtual inspection plays a storyboard, not a video.

What is untested: the Claude path in the copilot (no API key was available during the build); the scripted fallback is what runs by default.

- [ ] Test the copilot with an ANTHROPIC_API_KEY and tune the system prompt on three files
- [ ] Replace the synthetic affidavit, faculty list and CA certificate with two or three real sample PDFs run through OCR and extraction
- [ ] Wire one real registry in sandbox (UDIN verification is the simplest public one) and show a live response on the document page
- [ ] Move the data layer behind an API so the pages read from a database instead of the seed
- [ ] Rehearse the five-minute walkthrough with the team and time it
- [ ] Decide the theme direction (Blueprint, Registry Ledger or Carbon Copy) and apply it across every page
- [ ] Add Hindi labels for the pre-check and the deficiency letter